claude-windows / 78dabe12-41fd-46c2-ae90-ab74bdaec3c6

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\78dabe12-41fd-46c2-ae90-ab74bdaec3c6\subagents\workflows\wf_18dcd7ff-5ae\agent-ab369e61d316c0c4c.jsonl`
- SHA-256: `6a5fdf5a267af940342d2ff8e317175d527ce4d7e661ccf48d031c3d65b23b1a`
- Source modified: `2026-06-22T10:27:13+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_18dcd7ff-5ae`
- session_id: `78dabe12-41fd-46c2-ae90-ab74bdaec3c6`

Transcript

1. user (2026-06-22T10:25:19.415Z)

CONTEXT ? a GCS corpus-migration work item just completed on branch work/golden-corpus-hygiene (PR #316,
repo Khelsutra/badminton-highlight-indexer). Goal: run ALL training+inference from the cloud with ALL data
in gs://khelsutra-rally-corpus so the local Windows box + external drives stop being blockers.

AUTHORITATIVE DOC TO REVIEW: docs/DATA_IN_GCS.md (the corpus source-of-truth map).
RELEVANT CODE: backend/worker/__main__.py (cmd_infer ~157, cmd_train ~332, --corpus-uri ~473), backend/storage/ref.py (URI schemes/resolve_input_uri), deploy/gcp/README.md, docs/SETUP_NEW_MACHINE.md,
docs/archives/past_projects/DECOUPLED_COMPUTE/.

VERIFIED BUCKET STATE (already confirmed deterministically this session ? treat as ground truth, do NOT
re-run gcloud, it may not be authed for you):
- trajectories/ = 15 objects (<name>.csv)
- labels/ = 15 canonical objects only, named <YYYY-MM-DD>_<name>.rallies.csv (date = earliest mtime
  across C:/F: copies); the 7 old inconsistent (*_proxy / bare GX010128,mahadevpura-1) were MD5-verified
  byte-identical dupes and DELETED.
- manifests/golden.json = 15 entries, every trajectory+label gs:// ref RESOLVES (0 dangling).
- videos/ = mahadevpura_1.mp4 (523MB, demo of Drive->bucket) + 3 pre-existing (GX030094_proxy.mp4,
  mahadevpura-1.MP4, mahadevpura_smoke_180s.mp4). 14 of 15 golden videos NOT yet uploaded.
- worker train reads labels/(+videos/) and BUILDS the manifest on-box (does NOT read manifests/golden.json);
  manifests/golden.json is for EVAL. trajectories are a cache (training can regenerate via --trajectory-source detector).
- Open: Phase-2 videos (7 Drive proxies + 8 large F: full-MP4s, proxy-first), Phase-3 flip eval/litmus to GCS.
- Sibling threads (from memory): a golden-fixtures P3 thread needs these same golden traj+label CSVs;
  the served Gate-A litmus (#236, merged) is pinned to local paths; original-6 labels turned out recoverable from F:.

LENS = DOC ACCURACY. Read docs/DATA_IN_GCS.md end-to-end and verify every load-bearing claim against the actual repo code (cite file:line). Especially: do the worker infer/train commands + flags exist as documented? Is the "worker train builds the manifest on-box, does not read manifests/golden.json" claim correct (read backend/worker/__main__.py)? Are the storage URI scheme claims correct (backend/storage/ref.py)? Does the doc overclaim anything vs the VERIFIED BUCKET STATE? Flag any inaccuracy.

Return ONLY the structured verdict.

2. assistant (2026-06-22T10:25:22.628Z)

I'll review docs/DATA_IN_GCS.md against the actual repo code. Let me start by reading the doc and the relevant code files.

3. user (2026-06-22T10:25:23.574Z)

```
1       # Data in GCS ? the corpus source-of-truth map
2
3       **Status:** LIVE / in-progress (migration not complete). **Owner-gated** decisions in
§7.
4       **Last inventoried:** 2026-06-22 against `gs://khelsutra-rally-corpus`.
5       **Goal (owner, 2026-06-22):** run **all** training + inference from the cloud with **all
data in GCS**,
6       so the local Windows box (and the intermittently-mounted external drive) is **no longer
a blocker**.
7
8       > **? Other sessions: this is where the data lives.** Pull from `gs://khelsutra-rally-
corpus`, not
9       > from anyone's `C:` / `F:` / Google Drive. If something you need isn't in the bucket
yet, it's in the
10      > **gap table (§5)** ? flag it / migrate it, don't hard-code a local path.
11
12      This doc is the **data map**. The *machine* runbooks are
[`SETUP_NEW_MACHINE.md`](SETUP_NEW_MACHINE.md)
13      (spin up a box) and [`deploy/gcp/README.md`](../deploy/gcp/README.md) (provision GPU +
train/infer).
14      The cloud engine itself was built by the (archived) DECOUPLED_COMPUTE project
15      (`docs/archives/past_projects/DECOUPLED_COMPUTE/`); this doc does **not** redesign it ?
it records
16      *where the bytes are* and *what's still missing*.
17
18      ---
19
20      ## 1. The bucket
21
22      | | |
23      |---|---|
24      | Bucket | **`gs://khelsutra-rally-corpus`** (single corpus bucket) |
25      | Project | `gen-lang-client-0306026826` (billing *Khelsutra Billing*) |
26      | Region | `asia-south1` (Mumbai), UBLA, co-located with the GPU VMs |
27      | Auth | SA `rally-worker@gen-lang-client-0306026826.iam.gserviceaccount.com`
(`roles/storage.objectAdmin`); key at `~/.gcp/rally-worker-sa.json` (local only, **never
committed**). Interactive: `gcloud auth login`. |
28      | Size today | ~2.06 GB (mostly past dry-run reels/outputs ? **not** the full corpus) |
29
30      The engine is **backend-agnostic by URI** (`backend/storage/ref.py`): `gcs://` (or
`gs://`),
31      `s3://`, `gdrive://`, or a bare local path. Pick the backend with **config + creds, no
code change**.
32
33      ---
34
35      ## 2. Canonical layout (the target contract)
36
37      The cloud worker's input/output contract (`backend/worker/__main__.py`; lineage:
38      `DECOUPLED_COMPUTE/04`):
39
40      | Prefix | Holds | Who reads/writes it |
41      |---|---|---|
42      | `videos/` | source videos / proxies (`<clip>.mp4`) | **input** to `worker infer` &
`worker train` |
43      | `labels/` | golden rally labels (`<clip>.rallies.csv`) | **input** ground truth to
```

```
`worker train` + all eval |
44      | `trajectories/` | cached WASB trajectory CSVs | **(PROPOSED ? see §7.2)** GPU-free
eval/litmus input |
45      | `weights/<version>/` | trained Gen-0 bundles (`weights.json` + a run `manifest.json`)
| **output** of `worker train` |
46      | `models/` | upstream pretrained weights (`wasb_badminton_best.pth.tar`) | input to the
WASB detector |
47      | `outputs/<video_id>/` | per-video inference outputs (intervals, etc.) | **output** of
`worker infer` |
48      | `highlights/`, `promo/` | rendered reels + promo artifacts | output / demo |
49
50      > **Note on the manifest.** `worker train` does **not** read a pre-staged corpus
`manifest.json` ? it
51      > *builds* one on-box at train time from `labels/` (+`videos/`), extracting trajectories
via the native
52      > detector (`--trajectory-source detector`). A `manifest.json` only appears as an
**output** inside each
53      > `weights/<version>/` bundle. The **eval/litmus** path is what wants a stable, path-
resolvable manifest
54      > (§7.3).
55
56      ---
57
58      ## 3. What's actually in the bucket today (2026-06-22)
59
60      > **Phase-1 landed 2026-06-22:** added `trajectories/` (15), 15 canonical
`labels/<date>_<name>.rallies.csv`,
61      > and `manifests/golden.json`. The table below is the **pre-Phase-1** snapshot + the
*remaining* gaps
62      > (videos, the 7 stale label objects to delete) ? see §7/§8.
63
64      | Prefix | Present | Drift / notes |
65      |---|---|---|
66      | `labels/` | 7 files: `GX010128`, `mahadevpura-1`, + `Badminton BXH 2_proxy`, `Boxhill
Doubles_proxy`, `GX020094_proxy`, `GX030094_proxy`, `mahadevpura-2_proxy` | ? **inconsistent
naming** (`_proxy` on some, not others); ? **missing** the original-6 labels + `GX010137`,
`GX010141` |
67      | `videos/` | 3: `GX030094_proxy.mp4`, `mahadevpura-1.MP4`, `mahadevpura_smoke_180s.mp4`
| ? corpus videos/proxies almost entirely absent |
68      | `trajectories/` | ? | ? **does not exist** (every trajectory is local-only) |
69      | `models/` | `wasb_badminton_best.pth.tar` | ? (? WASB-C3 weight-provenance still an
open legal item before any external share) |
70      | `weights/` | `0.1.0-l4/` | ? first real L4 train bundle |
71      | `outputs/` | one `<video_id>/`, `parity/` | ? dry-run outputs |
72      | `highlights/`, `promo/` | reels + promo | ? |
73
74      **Drift summary:** the bucket has extra prefixes (`highlights/`, `promo/`, `models/`
alongside
75      `weights/`) and **inconsistent `labels/` naming**, and is **missing the bulk of the
corpus inputs**
76      (`videos/`, all `trajectories/`, most `labels/`). It reflects past *dry runs*, not a
complete corpus.
77
78      ---
79
80      ## 4. How the cloud consumes it
81
82      Provisioning + the verified commands live in
[`deploy/gcp/README.md`](../deploy/gcp/README.md) §3?§5.
83      The data-facing entry points (`backend/worker/__main__.py`):
84
85      ```bash
86      # config (no secrets in config; auth via GOOGLE_APPLICATION_CREDENTIALS):
87      #   { "storage": { "backend": "local", "gcs": { "project": "gen-lang-client-0306026826"
} } }
```

```
# INFERENCE on one cloud video -> outputs/<video_id>/ in the bucket
python -m backend.worker infer \
  --video-uri gcs://khelsutra-rally-corpus/videos/<clip>.mp4 \
  --out-uri   gcs://khelsutra-rally-corpus --segmenter wasb_hybrid --config config.json

# TRAIN from the bucket (reads labels/ [+ videos/], extracts trajectories, builds manifest on-box,
# runs training.gen0.harness, pushes weights/<version>/):
RALLY_DETECTOR_IMPL=native python -m backend.worker train \
  --corpus-uri gcs://khelsutra-rally-corpus --out-uri gcs://khelsutra-rally-corpus \
  --version <v> --trajectory-source detector --segmenter wasb_hybrid --config config.json
```

A bare/relative path resolves under a configured `input_root`/`input_backend` (`backend/storage/ref.py::resolve_input_uri`), so workflows can be written bucket-relative.

---

5. The gap ? what's still local-only (why the box is still a blocker)

All of this is **tiny except the originals** ? the metadata that blocks cloud eval is < 25 MB total:

Data	Where it is now	Size	In bucket?	Needed for
20 WASB trajectory CSVs	`C:\?\Annotation Setup\Collect\Trajectories\`	**17.5 MB**	? none	eval/litmus (cached), train (`--trajectory-source cached`)
golden manifest	`output/human_lovo_manifest.json` (C:-absolute paths)	4.5 KB	?	eval corpus definition
original-6 labels	`F:\[Khelsutra]?\Training\Golden Labelled\*.rallies.csv`	~12 KB	?	train + eval
7 new-clip labels	repo `output/*.rallies.csv` (gitignored) **+** F: archive (backed up 2026-06-22)	~12 KB	? partial/inconsistent	train + eval
source videos / proxies	F: archive + Google Drive golden folder	**100s of GB**	? 3 only	infer + train (real WASB extraction)

> The corpus *metadata* (trajectories + labels + manifest, ~25 MB) is what actually blocks GPU-free
> cloud eval and the served Gate-A litmus ? and it's trivially small to migrate. The **videos** are the
> only large migration, and they already exist on **Google Drive** (the golden-corpus folder), so they
> can go **Drive ? bucket from a cloud box**, keeping the local machine out of the path entirely.

---

6. Migration plan (close the gap)

Phase 1 ? corpus metadata (?25 MB, do first, removes the eval blocker). Stage all trajectories,
all labels, and a **bucket-relative eval manifest** into `gs://khelsutra-rally-corpus`. Pending the §7
naming decision. Sketch:

```bash
# trajectories (decision-free; absent today)
gcloud storage cp "C:\?\Collect\Trajectories\*.csv" gs://khelsutra-rally-corpus/trajectories/
# labels (after the §7.1 naming decision) -> labels/<canonical-name>.rallies.csv
# eval manifest with gs:// (or bucket-relative) paths -> a stable manifest object (§7.3)
```

137
138   **Phase 2 ? videos ? `videos/<name>.mp4`** (bucket name = the eval `name`, consistent
with
139   `trajectories/` + `labels/`). Sources (resolved 2026-06-22 ? see
`scratch/copy_videos_to_gcs.ps1`):
140
141 | Source | Clips | Size |
142 |---|---|---|
143 | Drive golden folder `?/Golden Label Videos Request - June 15/[Done] Video N/`
(**proxies**) | `mahadevpura_1/2`, `GX010128`, `GX020094`, `GX030094`, `Badminton_BXH_2`,
`Boxhill_Doubles` (7) | ~13.5 GB |
144 | F: archive (**full MP4s** ? proxy-first before upload) | `GX010137`, `GX010141`, +
original-6 (`adarsh_avi_singles`, `kushagra_singles`, `largetest_doubles`, `gbaaddy`,
`testlarge_short`, `mahadevpura_singles`) (8) | ~50 GB |
145
146   **Two ways to run it:**
147   - **Decoupled (preferred ? no local box):** `rclone` on a cloud VM with a Google-Drive
remote ?
148   `rclone copy gdrive:"<?/[Done] Video N>/<proxy>.mp4" :gcs:khelsutra-rally-
corpus/videos/<name>.mp4`
149   (one-time owner step: `rclone config` ? `gdrive` remote OAuth). Or `gdown <file-id>`
on the box for
150   the shared folder. This is the path that actually removes the local box.
151   - **Interim (today, owner box ? streams Drive/F: ? local ? GCS):** `pwsh
scratch/copy_videos_to_gcs.ps1
152   -Execute` (dry-run without `-Execute`; `-Only <name>` for one; skip-existing). Uses
local bandwidth.
153
154   **Priority:** the 7 Drive **proxies** first (already 1080p, sufficient for detection).
The 8 large F:
155   full MP4s should be **proxied to 1080p first** (cheaper storage + faster cloud decode)
before upload.
156   ? Two pre-existing non-canonical video objects (`videos/GX030094_proxy.mp4`,
`videos/mahadevpura-1.MP4`)
157   are superseded once `videos/GX030094.mp4` / `videos/mahadevpura_1.mp4` land ? verify-
dupe-then-delete
158   (same as the label cleanup). Track per-clip in §8.
159
160   **Phase 3 ? flip eval/training to read GCS by default.** Point the eval manifest +
litmus at the
161   bucket (env override or `input_root`) so a fresh box/CI needs no local corpus. Re-verify
the suite +
162   the served Gate-A litmus read from `gcs://` (and re-measure if the corpus set changes ?
see PR #236).
163
164   ---
165
166   ## 7. Decisions
167
168   1. **`labels/` naming ? LOCKED (owner, 2026-06-22):** `labels/<YYYY-MM-
DD>_<name>.rallies.csv`, where
169   `<name>` is the eval/manifest clip name and `<YYYY-MM-DD>` is the label's **authoring
date** =
170   the *earliest* mtime across all known copies (C: working copy + F: archive ? robust
to copy-resets;
171   a restored C: copy shows today, the F: archive keeps the real date). Fixes the
`_proxy` inconsistency
172   and disambiguates recycled GoPro names by date. *(Future, when videos are in the
bucket: optionally
173   add a `video_id`/MD5 sidecar for the strongest keying ? deferred, needs hashing each
video.)*
174   2. **Cache `trajectories/` ? YES, done 2026-06-22.** `trajectories/<name>.csv` cached so
cloud/CI eval +
175   the served Gate-A litmus run **GPU-free** (training can still regenerate via
`--trajectory-source detector`).

```
    3. **Eval manifest in GCS ? done 2026-06-22.** `gs://khelsutra-rally-
corpus/manifests/golden.json`
       (15 clips, `gs://` trajectory + label paths). The **local** litmus keeps its local-
path manifest;
       flipping the litmus to read the GCS manifest is Phase 3.
    4. **Originals migration scope ? OPEN** ? proxies-only vs full 4K, and how much of the
15-clip golden
       set vs the wider archive. Source = Google Drive ? bucket from a cloud box (not the
local box).

    > Phase-1 was run via a dry-run planner (resolve every clip's trajectory + label, stamp
the label's
    > authoring date, build the `gs://` manifest) ? `gcloud storage cp` (planner kept in
gitignored
    > `scratch/`; promote it to `scripts/` if corpus-sync recurs).
    > ? **Cleanup DONE 2026-06-22:** the 7 pre-existing inconsistent label objects
(`*_proxy.rallies.csv`,
    > bare `GX010128`/`mahadevpura-1`) were confirmed **byte-identical (MD5)** to the new
canonical names
    > (content also on C:/F:) and **deleted** ? `labels/` now holds exactly the 15 canonical
objects.

    > ? **gcloud gotcha (Phase-2):** `gcloud storage cp` treats `[...]` as a glob, so source
paths under
    > `[Done] Video N/` (Drive) or `[Khelsutra] ?` (F:) fail with "matched no objects." The
owner-box
    > script (`scratch/copy_videos_to_gcs.ps1`) stages such paths through a bracket-free
temp file; the
    > `rclone` cloud path has no such issue.

    ---

    ## 8. Progress tracker

    | Item | Status |
    |---|---|
    | Bucket + SA + provisioning (deploy/gcp) | ? built |
    | Cloud worker `infer`/`train` by URI | ? built (`backend/worker`) |
    | This data-map doc | ? 2026-06-22 |
    | Phase 1: trajectories ? `trajectories/` (15) | ? 2026-06-22 |
    | Phase 1: labels reconciled ? `labels/<date>_<name>` (15) | ? 2026-06-22 |
    | Phase 1: eval manifest ? `manifests/golden.json` | ? 2026-06-22 |
    | Phase 1: remove 7 stale `_proxy`/bare label objects | ? 2026-06-22 (MD5-verified
dupes, deleted) |
    | Phase 2: corpus videos/proxies ? `videos/<name>.mp4` | ? mapping + script done;
**1/15** demo'd (`mahadevpura_1`); 14 remain ? run `rclone` on a cloud box (preferred) or the
owner-box script |
    | Phase 2: proxy the 8 large F: full-MP4 clips before upload | ? |
    | Phase 3: eval/litmus default to the GCS manifest | ? |

    **Definition of done:** a fresh GPU box (or CI) can run inference, the eval suite, and
Gen-0 training
    reading **only** from `gs://khelsutra-rally-corpus` ? no `C:` / `F:` / local-Drive path
required.

    ---

    ## Related
    - [`SETUP_NEW_MACHINE.md`](SETUP_NEW_MACHINE.md) ? spin up a box (storage backends in
§7).
    - [`deploy/gcp/README.md`](../deploy/gcp/README.md) ? GCP GPU provisioning + train/infer
(§5).
    - [`VIDEO_LOCALITY_MODEL.md`](VIDEO_LOCALITY_MODEL.md) ·
[`PER_VIDEO_WORKSPACE.md`](PER_VIDEO_WORKSPACE.md) ? locality tiers + `outputs/<video_id>/`.
    - `docs/archives/past_projects/DECOUPLED_COMPUTE/` ? the cloud-engine design lineage.
```

221

4. user (2026-06-22T10:25:24.382Z)

```
1       """Storage value objects + URI parsing for the pluggable storage layer.
2
3       A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5           <bare path>                 -> local   (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6           local://<path>              -> local   (local:///abs or local://rel/p)
7           s3://<bucket>/<key>         -> s3       (AWS + R2 + DO Spaces + MinIO via
endpoint_url)
8           gcs://<bucket>/<key>        -> gcs      (gs:// accepted, normalized to gcs://)
9           gdrive://<path>             -> gdrive   (path under your locally-MOUNTED Google
Drive "My Drive")
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14      from __future__ import annotations
15
16      import re
17      from dataclasses import dataclass
18      from datetime import datetime
19      from typing import Optional
20
21      _SCHEME_RE = re.compile(r"^([A-Za-z][A-Za-z0-9+.\-]*)://")
22      _KNOWN = ("local", "s3", "gcs", "gdrive")
23
24
25      @dataclass(frozen=True)
26      class StorageStat:
27          """Lightweight object metadata (uniform across backends)."""
28          size: int
29          modified: Optional[datetime] = None
30          etag: Optional[str] = None
31          content_type: Optional[str] = None
32
33
34      @dataclass(frozen=True)
35      class StorageRef:
36          """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
37          empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
38          backend: str
39          bucket: str
40          key: str
41          uri: str
42
43          @property
44          def name(self) -> str:
45              return self.key.rstrip("/").rsplit("/", 1)[-1]
46
47
48      def parse_uri(uri: str) -> StorageRef:
49          """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
50
51          A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
52          have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
53          """
54          s = str(uri)
```

```
55          m = _SCHEME_RE.match(s)
56          if not m:
57              return StorageRef("local", "", s, f"local://{s}")
58          scheme = m.group(1).lower()
59          rest = s[m.end():]
60          if scheme == "gs":
61              scheme = "gcs"
62          if scheme in ("local", "gdrive"):
63              # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My
Drive".
64              return StorageRef(scheme, "", rest, s)
65          if scheme in _KNOWN:
66              bucket, _, key = rest.partition("/")
67              return StorageRef(scheme, bucket, key, s)
68          raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")
69
70
71      def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "")
-> str:
72          """Resolve a user-supplied input reference into a storage URI
(COMPUTE_DECOUPLED_SERVING M2).
73
74          Precedence (first match wins):
75            1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
76            2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
local).
77            3. A bare/relative name + a non-empty ``input_root`` ? joined under it
78               (``"gs://bucket/videos"`` + ``"x.mp4"`` ? ``"gs://bucket/videos/x.mp4"``).
79            4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``"gcs"`` +
``"bucket/x"`` ?
80               ``"gcs://bucket/x"``).
81            5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
82
83          So the default (``input_backend="local"``, ``input_root=""``) is an identity
passthrough for every
84          path used today, preserving byte-for-byte behaviour.
85          """
86          s = str(raw)
87          if _SCHEME_RE.match(s):           # 1. explicit scheme wins
88              return s
89          # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
90          #    os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
91          #    "/data/x" is NOT absolute). Anchored == a leading "/" or "\\", or a Windows
drive prefix "X:".
92          if s.startswith(("/", "\\")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
93              return s
94          if input_root:                    # 3. join a relative name under the configured
root
95              return input_root.rstrip("/") + "/" + s.lstrip("/")
96          if input_backend and input_backend != "local":   # 4. prefix a bucket/key with the
declared backend
97              return f"{input_backend}://{s}"
98          return s                          # 5. bare/relative local path ? unchanged
99
```

5. user (2026-06-22T10:25:25.288Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
```

```
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
15      """
16      from __future__ import annotations
17
18      import argparse
19      import csv
20      import json
21      import logging
22      import os
23      import sys
24      import tempfile
25      from typing import Any, List
26
27      from backend import storage
28      from backend.compute import ComputeJobSpec, get_compute_backend
29      from backend.config import (
30          StartupCheckError,
31          enforce_startup_checks,
32          load_config,
33          probe_cuda_available,
34      )
35      from backend.infrastructure.database import Database
36      from backend.pipeline.segmenters.base import segmenter_registry
37      from backend.pipeline.validators.base import registry as validator_registry
38      from backend.reporting import emit_indexer_report
39      from backend.results import Failure
40      from backend.utils.ffmpeg import ffprobe_bin  # P2a: single ffmpeg/ffprobe resolver
41      from backend.utils.hashing import compute_video_id
42
43      logger = logging.getLogger("backend.worker")
44
45
46      def _coerce(v: str) -> Any:
47          """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
48          low = v.lower()
49          if low in ("true", "false"):
50              return low == "true"
51          for cast in (int, float):
52              try:
53                  return cast(v)
54              except ValueError:
55                  pass
56          return v
57
58
59      def _apply_overrides(config: Any, sets: List[str]) -> None:
60          """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
61          for kv in sets or []:
62              key, sep, val = kv.partition("=")
63              if not sep:
64                  raise ValueError(f"--set expects k=v, got {kv!r}")
```

```
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow([
                    f'{float(s.get("start_time", 0.0)):.3f}',
                    f'{float(s.get("end_time", 0.0)):.3f}',
                    s.get("shot_count", ""),
                    s.get("ending_reason", s.get("shot_count_confidence", "")),
                ])


    def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
        with open(path, "w") as f:
            json.dump({
                "video_id": video_id,
                "status": status,
                "segment_count": len(segments),
                "segments": [{"start": float(s.get("start_time", 0.0)),
                              "end": float(s.get("end_time", 0.0))} for s in segments],
            }, f, indent=2)


    def _run_startup_checks(config: Any) -> bool:
        """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
        best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
        cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
        (returns True, ZERO work) when no deployment.profile is selected.

        The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
        torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
        WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
        no-op of work, not just of output."""
        profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
        cuda = probe_cuda_available() if profile else None  # torch-free on the default-OFF
path
        try:
            enforce_startup_checks(config, cuda_available=cuda, logger=logger)
            return True
        except StartupCheckError:
            return False  # already logged at ERROR by enforce_startup_checks


    def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
        """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
        outputs are durably in the bucket (the backend bucket-polls the done-marker). The
```

```
dispatched
120         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
121
122         A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
123         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
124         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
125         from backend.infrastructure.execution_policy import PolicyTimeout
126
127         spec = ComputeJobSpec(
128             video_uri=args.video_uri, out_uri=args.out_uri,
129             segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
130         )
131         try:
132             result = compute.run_infer(spec)
133         except PolicyTimeout as e:
134             logger.warning("[worker] remote compute did not complete (no marker within
budget): %s", e)
135             return 4
136         logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
137                     result.video_id, result.returncode, result.outputs_uri)
138         return result.returncode
139
140
141     def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
int,
142                            status: str, storage_cfg: dict, scratch: str) -> None:
143         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
144         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
145         try:
146             marker = {"video_id": video_id, "returncode": returncode,
147                       "segment_count": segment_count, "status": status}
148             local = os.path.join(scratch, "_done.json")
149             with open(local, "w", encoding="utf-8") as f:
150                 json.dump(marker, f)
151             storage.put(local, done_uri, config=storage_cfg)
152             logger.info("[worker] wrote completion marker -> %s", done_uri)
153         except Exception as e:  # never fail a good run because the marker push hiccuped
154             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
155
156
157     def cmd_infer(args: argparse.Namespace) -> int:
158         config = load_config(args.config) if args.config else load_config()
159         _apply_overrides(config, args.set)
160         if not _run_startup_checks(config):
161             return 2
162
163         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
164         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
165         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
166         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
167         compute = get_compute_backend(config)
168         if compute is not None:
169             return _dispatch_remote(compute, args)
170
171         storage_cfg = config.storage.model_dump()
```

```
172
173         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
174         os.makedirs(scratch, exist_ok=True)
175         config.output.output_dir = scratch
176
177         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
178         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
179         print(f"[worker] pulled {args.video_uri} -> {local_video}")
180
181         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
182         video_id = compute_video_id(local_video)
183
184         # 3. VALIDATE (same registry as the main CLI).
185         val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
186         failures = [r for r in val_results if not r.passed]
187         db = Database(os.path.join(scratch, config.output.db_name))
188         db.add_video(video_id, local_video, "failed" if failures else "processed",
189                      [r.model_dump() for r in val_results])
190
191         segments: List[dict] = []
192         segmenter_actual = None
193         segmentation_ran = False
194         status = "failed"
195         try:
196             if failures:
197                 print("[worker] validation FAILED: "
198                       + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
199                 return 2
200             seg_name = args.segmenter or config.indexing.default_segmenter
201             segmenter_cls = segmenter_registry.get(seg_name)
202             if not segmenter_cls:
203                 print(f"[worker] unknown segmenter: {seg_name!r} "
204                       f"(have {list(segmenter_registry.list_available())})")
205                 return 2
206             segmenter_actual = seg_name
207             result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
208             segmentation_ran = True
209             if isinstance(result, Failure):
210                 print(f"[worker] segmenter FAILED: {result.message}")
211                 return 3
212             segments = result.value if hasattr(result, "value") else result
213             status = "success"
214             # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
215             # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
216             # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
217             if not segments:
218                 detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
219                                  or getattr(config.indexing, "detector_impl", None) or
"auto")
220                 logger.warning(
221                     "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
222                     "The detector substrate yielded no usable trajectories/windows ? check
the detector "
223                     "logs above; common causes: native runner UNAVAILABLE
(weights/repo/WASB_PYTHON env), "
224                     "the WASB env produced no detections, or the input resolution differs
```

```
from the tuned "
225                     "proxy resolution. Re-run with -v for the native command + frame
counts.",
226                     video_id, segmenter_actual, detector_impl,
227                 )
228         finally:
229             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
230             emit_indexer_report(
231                 db=db, video_id=video_id, video_path=local_video, config=config,
232                 val_results=val_results, val_context=val_context,
233                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
234                 was_reingest=False, segmentation_ran=segmentation_ran,
235             )
236
237         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
238         out_local = os.path.join(scratch, "outputs", video_id)
239         os.makedirs(out_local, exist_ok=True)
240         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
241         _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
242
243         out_prefix = args.out_uri.rstrip("/")
244         pushed = []
245         for fn in sorted(os.listdir(out_local)):
246             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
247             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
248             pushed.append(uri)
249
250         print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
251         for u in pushed:
252             print("   ", u)
253
254         # M6: a remote dispatcher passes --done-uri; write a completion marker so its
bucket-poll
255         # terminates (carries video_id ? the dispatcher's output path + the worker rc).
DEFAULT-OFF:
256         # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a
failed container
257         # job currently lets the dispatcher's bounded poll time out (a failure-marker is a
follow-up).
258         if getattr(args, "done_uri", None):
259             _write_done_marker(args.done_uri, video_id, 0, len(segments), status,
storage_cfg, scratch)
260         return 0
261
262
263     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
264     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
265     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
266
267
268     def _probe_video_fps_width(path: str):
269         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
270
271         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
272         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
273         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
274         inference report (it flags the fallback) but corrupting for REAL training data. So
```

```
here we
275         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
276         rather than guesses.
277         """
278         import json
279         import subprocess
280         try:
281             out = subprocess.check_output(
282                 [ffprobe_bin(), "-v", "error", "-select_streams", "v:0",
283                  "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
284                 stderr=subprocess.DEVNULL).decode()
285             st = (json.loads(out).get("streams") or [{}])[0]
286             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
287             fps = float(num) / float(den or "1")
288             width = float(st.get("width") or 0)
289             if fps > 0 and width > 0:
290                 return fps, width
291         except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
292             pass
293         return None
294
295
296     def _resolve_train_detector(args: argparse.Namespace, config: Any):
297         """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
298         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
299         stub=CPU mock). Returns the runner, or prints an error + returns None.
300
301         Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
302         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
303         no shuttle detector and is rejected.
304         """
305         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
306
307         seg_cls = segmenter_registry.get(args.segmenter)
308         if not seg_cls:
309             print(f"[worker] unknown segmenter: {args.segmenter!r} "
310                   f"(have {list(segmenter_registry.list_available())})")
311             return None
312         seg = seg_cls(None, config)
313         if not isinstance(seg, TrajectoryHybridSegmenter):
314             print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
315                   f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
316             return None
317         try:
318             runner, _ = seg._resolve_runner(config.get("indexing", {}))
319         except NotImplementedError as e:
320             # e.g. detector_impl='native' for a family without a native runner (tracknet, or
321             # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
322             print(f"[worker] detector not available: {e}")
323             return None
324         ok, msg = runner.healthcheck()
325         if not ok:
326             print(f"[worker] detector environment not ready: {msg}")
327             return None
328         print(f"[worker] detector ready ({args.segmenter}): {msg}")
329         return runner
330
331
```

```
    def cmd_train(args: argparse.Namespace) -> int:
        """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
        to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.

        Two trajectory sources (the train pipeline + harness are identical either way):
          * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
            shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
          * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
            DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
            is the M3.5 "first real training" path; only the trajectory source flips.
        """
        import subprocess

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
        labels_dir = os.path.join(scratch, "labels")
        traj_dir = os.path.join(scratch, "trajectories")
        videos_dir = os.path.join(scratch, "videos")
        os.makedirs(labels_dir, exist_ok=True)
        os.makedirs(traj_dir, exist_ok=True)

        corpus = args.corpus_uri.rstrip("/")
        label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
                            if u.lower().endswith(".csv"))
        if len(label_uris) < 2:
            print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
                  f"under {corpus}/labels/")
            return 2

        # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
        runner = None
        video_by_stem: dict = {}
        if args.trajectory_source == "detector":
            os.makedirs(videos_dir, exist_ok=True)
            runner = _resolve_train_detector(args, config)
            if runner is None:
                return 2
            for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
                vname = storage.parse_uri(vuri).name
                if not vname.lower().endswith(_VIDEO_EXTS):
                    continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
                video_by_stem[os.path.splitext(vname)[0]] = vuri

        print(f"[worker] trajectory source: {args.trajectory_source}")
        specs: List[dict] = []
        for uri in label_uris:
            fname = storage.parse_uri(uri).name                    # e.g.
GX020094_proxy.rallies.csv
            name = fname.replace(".rallies.csv", "").replace(".csv", "")
            local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
```

```
            if args.trajectory_source == "detector":
                vuri = video_by_stem.get(name)
                if not vuri:
                    print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
                    continue
                local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
                                            config=storage_cfg)
                video_id = compute_video_id(local_video)
                # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
                # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
                meta = _probe_video_fps_width(local_video)
                if meta is None:
                    print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
                    continue
                fps, frame_width = meta
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
                    continue
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": fps, "frame_width": frame_width})
            else:
                from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": args.fps, "frame_width": args.frame_width})

        if len(specs) < 2:
            print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [sys.executable, "-m", "training.gen0.harness",
               "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
        if args.floor:
            floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
                           if "://" in args.floor else args.floor)
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
```

```
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(prog="python -m backend.worker",
                                    description="Storage-aware worker: pull ? run pipeline ?
push.")
        p.add_argument("-v", "--verbose", action="store_true",
                       help="DEBUG logging (the native detector command, frame counts, per-
stage detail)")
        sub = p.add_subparsers(dest="cmd", required=True)

        pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
        pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
        pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
        pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
        pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
        pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
        pi.add_argument("--max-frames", type=int, default=None)
        pi.add_argument("--done-uri", default=None,
                        help="M6: storage URI for a completion MARKER (video_id + rc)
written on a "
                             "successful run, so a remote (Cloud Run) dispatcher's bucket-
poll terminates. "
                             "Default-OFF: omit it for the normal local run.")
        pi.add_argument("--set", action="append", default=[], metavar="k=v",
                        help="config override, e.g. --set indexing.skip_ai_handoff=true")
        pi.set_defaults(func=cmd_infer)

        pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
        pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
                        "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
        pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
        pt.add_argument("--version", required=True, help="semver for the artifact bundle")
        pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
                        help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
                             "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
                             "(needs videos/ in the corpus; honours detector_impl).")
        pt.add_argument("--segmenter", default="wasb_hybrid",
                        help="trajectory hybrid whose detector is used for --trajectory-
source detector")
        pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
        pt.add_argument("--config", default=None)
        pt.add_argument("--scratch", default=None)
        pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
```

```
487         pt.add_argument("--frame-width", type=float, default=1280.0,
488                         help="synth trajectory frame width (detector probes the video)")
489         pt.add_argument("--set", action="append", default=[], metavar="k=v")
490         pt.set_defaults(func=cmd_train)
491         return p
492
493
494     def _setup_logging(verbose: bool) -> None:
495         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
496         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
497         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
498         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
499         entry), not at import, so importing the module never reconfigures a caller's
logging."""
500         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
501         # configured logging at import), which would leave the level at WARNING and re-
suppress the
502         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
503         logging.basicConfig(
504             level=logging.DEBUG if verbose else logging.INFO,
505             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
506             datefmt="%H:%M:%S",
507             force=True,
508         )
509
510
511     def main(argv=None) -> int:
512         args = build_parser().parse_args(argv)
513         _setup_logging(getattr(args, "verbose", False))
514         return args.func(args)
515
516
517     if __name__ == "__main__":
518         sys.exit(main())
519
```

6. assistant (2026-06-22T10:25:29.359Z)

Now let me verify the storage layer functions referenced (`storage.list_uris`, `storage.fetch`, `storage.put`) and check the eval/litmus claims. Let me also check the deploy/gcp README and the served Gate-A litmus.

7. user (2026-06-22T10:25:30.908Z)

```
backend\storage\ref.py:48:def parse_uri(uri: str) -> StorageRef:
backend\storage\ref.py:71:def resolve_input_uri(raw: str, *, input_backend: str = "local",
input_root: str = "") -> str:
backend\storage\__init__.py:106:def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None,
*,
backend\storage\__init__.py:114:def put(local_path: "str | os.PathLike[str]", uri: str, *,
backend\storage\__init__.py:122:def list_uris(prefix_uri: str, *, config: Optional[Dict[str,
Any]] = None) -> Iterator[str]:
```

8. user (2026-06-22T10:25:31.336Z)

```
backend\storage\gcs.py
backend\storage\local.py
backend\storage\s3.py
```

```
backend\storage\base.py
backend\storage\gdrive.py
backend\storage\ref.py
backend\storage\__init__.py
```

9. assistant (2026-06-22T10:25:36.390Z)

```
Good ? `list_uris`, `fetch`, `put` all exist, and gcs/s3/local/gdrive backends exist. Now let me
verify the doc's specific claims about config structure (`storage.backend`,
`storage.gcs.project`), the `models/` weights path, and the eval/litmus pinned-to-local claim.
Let me check the config and the Gate-A litmus.
```

10. user (2026-06-22T10:25:38.082Z)

```
backend/config/models.py:65:    weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
Binary file backend/config/__pycache__/models.cpython-313.pyc matches
Binary file backend/detectors/__pycache__/wasb_infer.cpython-313.pyc matches
Binary file backend/detectors/__pycache__/wasb_runner.cpython-313.pyc matches
backend/pipeline/detectors/wasb_infer.py:36:    python wasb_infer.py --frames_dir /path/frames
--weights ../pretrained_weights/wasb_badminton_best.pth.tar \
backend/pipeline/detectors/wasb_runner.py:37:    weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
backend/pipeline/detectors/wasb_runner.py:58:                                "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
Binary file backend/pipeline/detectors/__pycache__/wasb_infer.cpython-313.pyc matches
Binary file backend/pipeline/detectors/__pycache__/wasb_runner.cpython-313.pyc matches
deploy/cloudrun/Dockerfile:56:    WASB_WEIGHTS=/opt/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
deploy/cloudrun/README.md:15:- The WASB weights staged in the bucket (WASB-C3 unresolved ?
owner-gated): `gs://<bucket>/models/wasb_badminton_best.pth.tar`.
deploy/cloudrun/README.md:31:  --set-env-vars WASB_WEIGHTS=/staged/wasb_badminton_best.pth.tar
deploy/digitalocean/gpu_setup.sh:52:WEIGHTS="$WASB_REPO/pretrained_weights/wasb_badminton_best.p
th.tar"
deploy/digitalocean/gpu_setup.sh:56:  echo "[gpu]       !! MISSING $WEIGHTS ? fetch
wasb_badminton_best.pth.tar (Drive/bucket) before extraction."
deploy/gcp/README.md:183:WASB weights (`wasb_badminton_best.pth.tar`) must be present at
deploy/gcp/README.md:185:Point the native runner at them: `export WASB_WEIGHTS=~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar`
```

11. user (2026-06-22T10:25:39.848Z)

```
backend\config\startup.py:47:# CLOUDSDK_CONFIG ? those are often set without usable creds (a
plain project id / a config dir
backend\config\models.py:311:class StorageConfig(BaseModel):
backend\config\models.py:318:    backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
backend\config\models.py:326:    # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An
explicit scheme on the input
backend\config\models.py:329:    input_backend: Literal["local", "s3", "gcs", "gdrive"] =
"local"
backend\config\models.py:344:    gcs: dict[str, Any] = Field(default_factory=dict)
```

12. assistant (2026-06-22T10:25:43.906Z)

```
The config claims check out: `storage.backend` (Literal local/s3/gcs/gdrive) and `storage.gcs`
(dict) both exist. Let me verify the config example in the doc more carefully ? it shows
`"backend": "local"` with a `gcs.project`. And let me check the served Gate-A litmus claim
(pinned to local paths) and the `models/` weights bucket path claim.
```

13. user (2026-06-22T10:25:45.350Z)

```
311     class StorageConfig(BaseModel):
312         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
313         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
```

```
Spaces
314         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
315         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
316         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
317         (boto3/google auth chains supply credentials). See ``backend/storage/``."""
318         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
319         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
320         output_root: str = ""
321         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
322         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
323         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
324         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
325         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
326         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
327         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
328         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
329         input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
330         input_root: str = ""
331         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
332         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
333         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
334         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
335         # the legacy flat layout is used unchanged.
336         single_folder_per_video: bool = False
337         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
338         workspace_root: str = ""
339         # Cloud namespace prefix under the root so per-video folders sit tidily beside
340         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
341         workspace_prefix: str = "workspaces"
342         local: dict[str, Any] = Field(default_factory=dict)
343         s3: dict[str, Any] = Field(default_factory=dict)
344         gcs: dict[str, Any] = Field(default_factory=dict)
345         gdrive: dict[str, Any] = Field(default_factory=dict)
346
347
348     class DeploymentConfig(BaseModel):
349         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
350         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
351
352         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
353         behaves exactly as before this section existed. A profile is opt-in (``config.json``
354         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
355         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
```

14. user (2026-06-22T10:25:46.316Z)

```
175     bash deploy/digitalocean/gpu_setup.sh                                          # native
`wasb` conda env + WASB-SBDT
176
177     # secrets (NOT in repo): the GCS SA JSON + a rotated Gemini key
178     gcloud compute scp ~/.gcp/rally-worker-sa.json rally-gpu:/root/.gcp/sa.json --zone=asia-
south1-a
179     # on the box: export GOOGLE_APPLICATION_CREDENTIALS=/root/.gcp/sa.json
180     mkdir -p /root/.keys && printf '%s' '<ROTATED_GEMINI_KEY>' > /root/.keys/GEMINI_API_KEY
&& chmod 600 /root/.keys/GEMINI_API_KEY
181     ```
182
183     WASB weights (`wasb_badminton_best.pth.tar`) must be present at
184     `~/models/WASB-SBDT/pretrained_weights/` (gpu_setup.sh does NOT fetch them ? stage from
Drive/bucket).
185     Point the native runner at them: `export WASB_WEIGHTS=~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar`
186     and `export WASB_PYTHON=$(conda run -n wasb which python)` (or the env's python path).
See
187     `docs/TRACKNET_WSL_SETUP.md` for the WASB env. **WASB-C3:** weight provenance is an open
legal item
188     before sharing any trajectories/model externally ? internal extraction + Gen-0 is fine.
189
190     ---
191
192     ## 5. Stage the corpus + run the first REAL train/infer
193
194     Config for GCS (no secrets in config ? auth via `GOOGLE_APPLICATION_CREDENTIALS`):
```

15. assistant (2026-06-22T10:25:50.789Z)

Now let me check two things the doc claims: (1) the deploy/gcp README §5 config example to compare with the doc's config snippet, and (2) the doc's `models/` bucket claim ? it says `models/wasb_badminton_best.pth.tar` is "input to the WASB detector". The detector actually reads from a local path (`~/models/...` or `WASB_WEIGHTS` env), staged from the bucket, not directly from the bucket. Let me check the deploy README §5 verified commands.

16. user (2026-06-22T10:25:52.028Z)

```
192     ## 5. Stage the corpus + run the first REAL train/infer
193
194     Config for GCS (no secrets in config ? auth via `GOOGLE_APPLICATION_CREDENTIALS`):
195
196     ```json
197     { "sport": "badminton",
198       "indexing": { "default_segmenter": "wasb_hybrid", "detector_impl": "native" },
199       "storage": { "backend": "local", "gcs": { "project": "gen-lang-client-0306026826" } }
}
200     ```
201
202     ```bash
203     # stage the 7 Done videos + 7 labels into the bucket (from the co-located box ? fast):
204     #   videos -> gs://khelsutra-rally-corpus/videos/   labels -> gs://khelsutra-rally-
corpus/labels/
205     # (owner handles the actual copy; see "Upload options" below)
206
207     # FIRST REAL training (real WASB trajectories, native runner):
208     RALLY_DETECTOR_IMPL=native python -m backend.worker train \
209       --corpus-uri gcs://khelsutra-rally-corpus --out-uri gcs://khelsutra-rally-corpus \
210       --version 0.1.0 --trajectory-source detector --segmenter wasb_hybrid --config
config.json
211
212     # FIRST REAL inference on the held-out Video 1 (GX010135_proxy.mp4, no labels):
213     python -m backend.worker infer \
214       --video-uri gcs://khelsutra-rally-corpus/videos/GX010135_proxy.mp4 \
215       --out-uri  gcs://khelsutra-rally-corpus --segmenter wasb_hybrid --config config.json
```

```
216     ```
217
218     ### Upload options (how to get a video in + run inference)
219     The worker resolves `--video-uri` via the storage layer ? pick whichever fits:
220     - **GCS bucket (recommended, co-located):** `gcs://khelsutra-rally-
corpus/videos/clip.mp4`. Upload with
221       `gcloud storage cp clip.mp4 gs://khelsutra-rally-corpus/videos/`.
222     - **S3 / DO Spaces / R2:** `s3://bucket/clip.mp4` (install `.[storage-s3]`, creds in
`~/.aws/credentials`).
223     - **Local upload:** scp the file to the box and pass a bare path /
`local:///root/clip.mp4` (identity passthrough, no copy).
224     - **Google Drive (assumes Drive is MOUNTED):** mount Google Drive for Desktop ? a fair
requirement to
225       use Drive files locally ? and reference them with `gdrive://<path-under-My-Drive>`.
The backend resolves
226       it against your local mount (auto-detected: Windows `G:\My Drive`, macOS
227       `~/Library/CloudStorage/GoogleDrive-<acct>/My Drive`; override with
`GDRIVE_MOUNT_ROOT` or
228       `storage.gdrive.mount_root`), then it's plain local I/O ? **no service account, no
API, no extra install.**
229       E.g. the golden-corpus folder `?/drive/folders/1sHGL7iacnmM80ChT-p2cKy638gK504T_`
mounts at
230       `G:\My Drive\Sharing\Annotation Videos\Golden Label Videos Request - June 15\`, so
231       `gdrive://Sharing/Annotation Videos/Golden Label Videos Request - June 15/[Done] Video
4 - Badminton/GX020094_proxy.mp4`
232       (or just the raw `G:\My Drive\?` path ? the `local` backend handles that too). ? A
**headless GPU VM has
233       no mount** ? there, stage Drive?bucket and read `gcs://`, or `gdown <file-id> -O
clip.mp4` to a local path.
234
235     ---
236
237     ## 6. Switch off between sessions (cost control)
238
239     > **? DELETE, don't just STOP.** A *stopped* VM halts compute billing but **still bills
for its
240     > persistent disk** (pd-ssd ? $0.17/GB/mo ? a 200 GB boot disk ? **$34/mo**). Only
**delete** reaches
241     > $0. So `teardown.sh` **defaults to `delete`** (owner cost rule, 2026-06-20). The boot
disk has
242     > `autoDelete=true`, so delete reclaims it too ? verified 2026-06-20.
243
244     ```bash
245     bash deploy/gcp/teardown.sh          # DELETE the VM (default; $0 ? removes VM + boot
disk)
246     bash deploy/gcp/teardown.sh delete   # same (explicit)
247     bash deploy/gcp/teardown.sh stop     # STOP only ? keeps the disk ? STILL BILLS
(~$/mo!); fast resume
248     bash deploy/gcp/teardown.sh all      # delete VM + SA + budget (keeps the bucket/data)
249     # cross-region VM: set the zone (teardown defaults to asia-south1-a)
250     RALLY_GCP_ZONE=asia-southeast1-a bash deploy/gcp/teardown.sh delete
251     ```
```

17. assistant (2026-06-22T10:26:00.275Z)

The doc's config snippet and worker commands match the deploy/gcp README exactly (verified source of truth). Now let me verify a few remaining load-bearing claims:

1. The doc §4 train comment says "extracts trajectories, builds manifest on-box" ? the code confirms `cmd_train` builds `manifest.json` in scratch (line 417-419), and reads `labels/` (line 359) + `videos/` (line 374), never reads `manifests/golden.json`. ?
2. The doc §2 note "worker train does not read a pre-staged corpus manifest.json ? it builds one on-box... A manifest.json only appears as an output inside each weights/<version>/ bundle." Let me verify the bundle output includes manifest.json.

The harness writes `weights.json` + `manifest.json` into the bundle. The doc claims the bundle contains "weights.json + a run manifest.json". Let me verify what the gen0 harness actually outputs.

18. user (2026-06-22T10:26:00.580Z)

```
3:    python -m training.gen0.harness --manifest output/human_lovo_manifest.json \
11:      <out>/<version>/weights.json    (gitignored; classifier + frozen normalization)
12:      <out>/<version>/manifest.json   (the reviewable contract: feature-schema version,
201:    weights_path = os.path.join(bundle_dir, "weights.json")
213:            "weights_file": "weights.json", "weights_sha256": _sha256(weights_path),
241:    manifest_out = os.path.join(bundle_dir, "manifest.json")
```

19. assistant (2026-06-22T10:26:07.767Z)

Confirmed: the gen0 harness writes `weights.json` + `manifest.json` into `<out>/<version>/`, exactly as the doc claims. The "output manifest.json in the bundle" claim is accurate.

Now let me verify the doc's §2 claim about `--trajectory-source detector` being the default for train-from-bucket. The doc §4 train example uses `--trajectory-source detector` and the §2 note says train "extracts trajectories via the native detector (`--trajectory-source detector`)". But the code default is `synth` (line 477). The doc's §2 note phrasing ("it builds one on-box at train time from labels/ (+videos/), extracting trajectories via the native detector (--trajectory-source detector)") describes the detector path specifically, which is fine since it's shown as the explicit flag. Let me check the served Gate-A litmus pinned-to-local claim and the original-6 labels recoverable claim against repo to confirm the doc isn't overclaiming.

20. user (2026-06-22T10:26:09.511Z)

```
backend/config/models.py
backend/config/startup.py
backend/eval/ablation.py
backend/eval/ablation_pdf.py
backend/eval/batch.py
backend/eval/calibrate_local.py
backend/eval/eval_partial_labels.py
backend/eval/experiment.py
backend/eval/export_wasb_segments.py
backend/eval/golden_fixtures.py
---human_lovo_manifest---
./output/human_lovo_manifest.json
```

21. assistant (2026-06-22T10:26:20.481Z)

The `output/human_lovo_manifest.json` exists (gitignored, C:-absolute paths), matching the doc's §5 gap-table claim. The doc's claims about the litmus being local-path-pinned are consistent with context (not the focus of this DOC ACCURACY lens for code-verifiable claims; it's a Phase-3 "still to flip" assertion).

Let me do a final check on one potential overclaim: the doc §2 table says `trajectories/` is "(PROPOSED ? see §7.2)" but §3's Phase-1 banner and §7.2 say trajectories were DONE 2026-06-22. That's an internal inconsistency in the doc. Let me re-read those exact spots.

The doc §2 table row: `trajectories/ | ... | (PROPOSED ? see §7.2) GPU-free eval/litmus input`. But §7.2 says "Cache trajectories/ ? YES, done 2026-06-22" and §3/§8 mark it DONE. So §2 labeling trajectories as "PROPOSED" is stale/contradictory with the rest of the doc and with the VERIFIED BUCKET STATE (15 objects present). This is a real doc inaccuracy.

Let me also check whether the worker `train --trajectory-source detector` actually keys videos?labels by stem correctly given the doc's new canonical label naming `<date>_<name>.rallies.csv`. The code at line 384 derives `name` by stripping `.rallies.csv`, so for `2026-06-22_GX020094.rallies.csv` the `name` becomes `2026-06-22_GX020094`, and it looks up `video_by_stem[name]` ? which would need the video named `2026-06-22_GX020094.mp4`. But the doc Phase-2 says videos go to `videos/<name>.mp4` where name = `GX020094` (no date). This is a real

keying mismatch worth flagging.

22. user (2026-06-22T10:26:21.707Z)

```
381         specs: List[dict] = []
382         for uri in label_uris:
383             fname = storage.parse_uri(uri).name                    # e.g.
GX020094_proxy.rallies.csv
384             name = fname.replace(".rallies.csv", "").replace(".csv", "")
385             local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
386             if args.trajectory_source == "detector":
387                 vuri = video_by_stem.get(name)
388                 if not vuri:
389                     print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
390                     continue
391                 local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
392                                             config=storage_cfg)
393                 video_id = compute_video_id(local_video)
394                 # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
```

23. assistant (2026-06-22T10:26:31.012Z)

Confirmed the keying mismatch. The worker derives `name` from the label filename by stripping only `.rallies.csv`/`.csv` ? so the new canonical label `<date>_<name>.rallies.csv` yields `name = "2026-06-22_GX020094"`, and it looks up the video by that exact stem (`video_by_stem.get(name)`). But the doc's Phase-2 plan stages videos as `videos/<name>.mp4` with `name` = bare clip name (e.g. `GX020094.mp4`, `mahadevpura_1.mp4`), with no date prefix. The stems won't match ? every video would be "no video for ... skipping" ? `--trajectory-source detector` train can't run from the bucket as the doc describes.

This is a genuine, load-bearing inaccuracy: the doc's §7.1 LOCKED label naming + Phase-2 video naming are mutually incompatible with the worker's `cmd_train` stem-join (and the doc never notes the discrepancy). I have enough to deliver the verdict.

24. user (2026-06-22T10:27:03.214Z)

Structured output provided successfully

25. assistant (2026-06-22T10:27:13.624Z)

Verdict delivered via StructuredOutput. Summary of what I verified against code:

- **Accurate:** worker `infer`/`train` subcommands + all documented flags exist (`backend/worker/__main__.py:457-490`); URI-scheme/backend-agnostic claims hold (`backend/storage/ref.py`; gcs/s3/local/gdrive backends + `list_uris`/`fetch`/`put` present); config snippet matches `deploy/gcp/README.md:199`; and the central "train builds the manifest on-box, does NOT read `manifests/golden.json`" claim is **correct** (`cmd_train` reads `labels/`+`videos/`, writes `manifest.json` to scratch; `manifest.json` appears only as a `weights/<version>/` bundle output per `training/gen0/harness.py:241`).

- **1 blocker:** the §7.1-LOCKED dated label naming (`<date>_<name>.rallies.csv`) is incompatible with the worker's video?label stem-join (`backend/worker/__main__.py:384,387`) and with the doc's own Phase-2 bare-named videos ? so `--trajectory-source detector` train-from-bucket can't actually run as documented, and the doc never flags it.

- **1 major:** §2 contradicts the rest of the doc + the verified bucket by tagging `trajectories/` as "PROPOSED" when it's DONE (15 objects present).

- **Minor/nits:** §2 implies `models/` weights are read from the bucket directly (they're staged to a local path); §2 note reads as if `--trajectory-source detector` is the default (default is

`synth`); and the label-count framings across §3/§5/§8 are hard to reconcile on one read though the numbers check out.